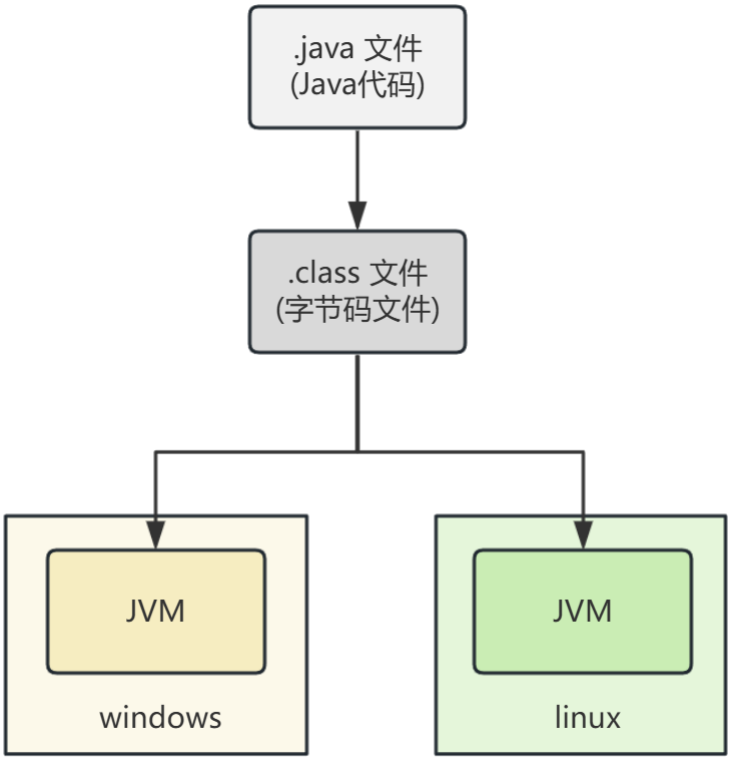
JVM基础

JVM到底是个啥?

其实就是一个Java程序的运行环境. 我们写的Java代码, 一编译就变成了.class文件, 这就是Java的二进制字节码文件, 把.class文件给jvm, jvm就能帮你运行了.

为啥jvm能跨平台呢?

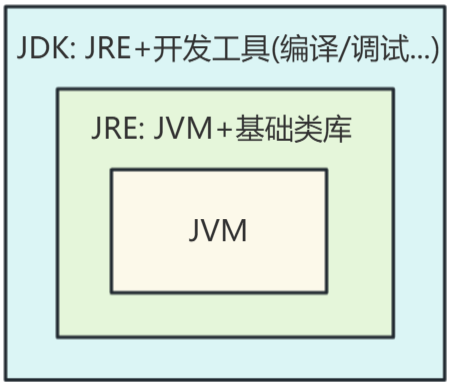


本质上就是不同的平台, 或者说不同的操作系统上有不同版本的jvm, 就是适用于对应操作系统的jvm. 这些jvm都能运行java代码编译后的.class文件, 这个.class文件都是相同的.

然后.class文件被不同操作系统上的jvm解释执行, 不同操作系统的jvm就能把.class文件解释为对应的操作系统能运行的指令, 所以java就能跨平台了. 这就是所谓的一次编译, 处处运行.

但是docker出现后, java这种跨平台的优势就被大大削弱了.

那JVM, JRE, JDK都有啥关系呢?

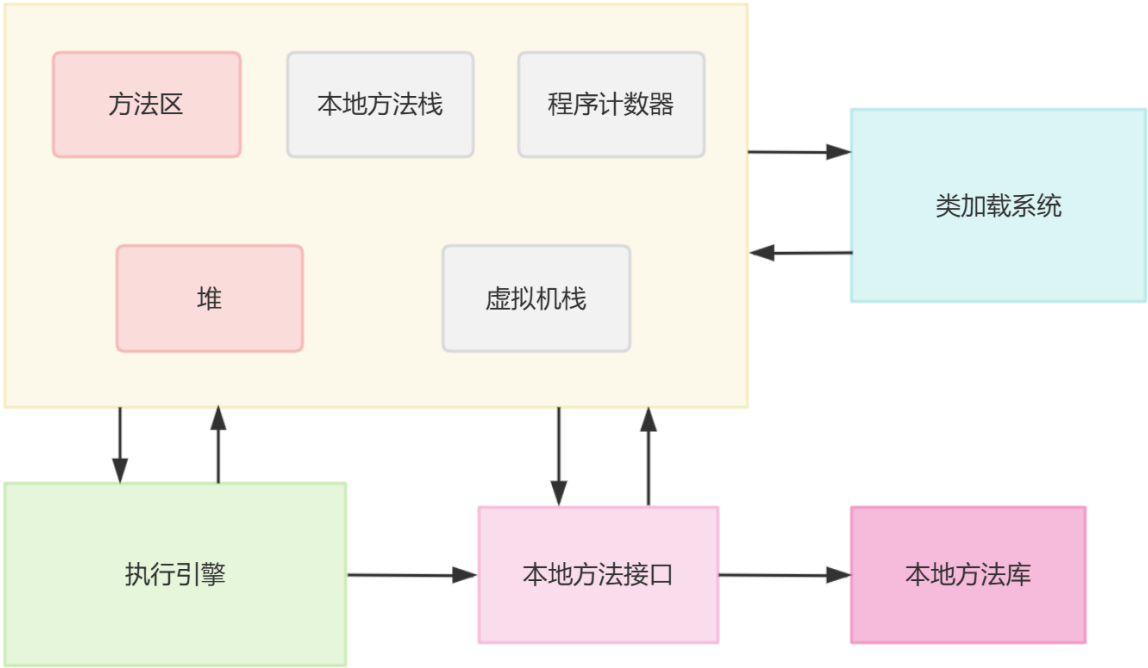


JVM刚说了是运行.class文件的一个虚拟机.

JRE就等于JVM+基本类库. 因为代码执行需要调用很多JavaSE的类库, 所以JRE就在JVM的基础上带了这些类库.

但是我们不是一上来就写.class文件的, 我们写的是Java代码, Java代码需要被编译为.class文件, 谁来编译? JDK就等于JRE+编译调试打包工具.

那么问题来了, jvm咋帮你运行程序的呢?



首先Java代码, 必须要被编译为.class文件才能被jvm运行.

那JVM大概能分为这个几个部分, 类加载系统, 运行时数据区, 执行引擎, 本地方法接口+本地方法库

类加载系统

- 首先我们的.class文件里面有各种各样的类, 它是怎么被加载运行的呢?
- 就是通过jvm中的类加载系统给加载到运行时数据区.
- 运行时数据区听名字就知道是一块内存区域, 是放代码运行时产生的数据的.
- 严格来说类被加载之后会被放到运行时数据区中的方法区, 方法区就是专门放已加载的类的元数据信息的.
- 一个类的类名、父类、接口、字段、方法、访问修饰符等等都是放这儿的.

运行时数据区

- 代码执行的时候, 会new对象, new出的对象一般会放到运行时数据区的堆区. 然后代码执行肯定要调用方法的, 方法会被放到运行时数据区的栈里面, 就是这个虚拟机栈.
- 那么问题来了, 代码一行一行被执行, 有时候多个线程同时执行, 然后各个方法各个类跳来跳去的, 怎么知道执行到哪儿了呢? 这个程序计数器就是用来记录代码执行到哪儿了.
- 最后还有这个本地方法栈是干啥的? 本地方法就是C/C++写的方法, java底层需要调用很多C/C++写的方法, 调这些本地方法就会被放到本地方法栈.

执行引擎

运行时数据区就是几块内存空间, 这些字节码到底是怎么执行的呢?

- 注意, 字节码文件只是 JVM 的一套指令集规范，不能直接交给底层操作系统去执行. 所以就需要执行引擎将字节码解释为底层系统指令, 再交由 CPU 去执行.
- 不同操作系统的jvm把字节码指令解释为对应的操作系统能运行的指令, 这就是Java能跨平台的关键.
- 注意这里说的是解释执行. Java编译器将Java文件编译为字节码文件, jvm执行引擎把字节码指令解释为对应操作系统能运行的指令, 解释执行 (其实这里还有jit即时编译, 比较复杂就不展开聊了). 这也就是为什么说Java是编译和解释并存的语言.

本地方法接口 本地方法库

最后, Java代码执行的时候需要调用底层C/C++写的本地方法, 所以这里就需要本地方法接口和本地方法库做交互. 刚说了, 本地方法执行的时候, 会放到运行时数据区的本地方法栈.

这就是jvm执行代码的基本过程.